

BBO Capstone Project

Model Card

Table of Contents

Overview2

Intended use2

Details2

 Week 1:2

 Week 2:2

 Week 3:3

 Week 4:3

 Week 5:3

 Week 6:4

 Week 7:4

 Week 8:5

 Week 9:5

Performance:5

Assumptions and limitations:6

Ethical considerations:6

Overview

There are eight synthetic functions in the Black Box Optimisation (BBO) Capstone project. Each function is designed to mimic real-world complexity, with features such as non-linearity and multiple local maxima.

The Bayes Optimisation for BBO model builds a surrogate model using the initial input data that is provided and attempts to find the point/query that returns the optimum value for the real function.

Intended use

The model is used for predicting the next point to evaluate against the real function. The model works towards maximising the real functions.

The model is a config driven, hybrid surrogate-based optimiser and is not a general purpose LLM. The hybrid nature of the model works with high dimensions but does not solve the curse of dimensionality.

Details

The model, in general, uses a surrogate model and Bayesian Optimisation to identify best candidates to run against the real function each week. My method of evaluation of the candidate points has evolved over time.

Week 1:

I have used the Gaussian Process Regressor as the surrogate for the real function. I have tried different acquisition functions, PI, EI, UCB and since the ask is to maximise, I have picked the points with the maximum value.

As the dimensions of the input increase, I have tried using different values for exploration parameter (κ). It also tried using PI and EI with different values of the exploration bias (ξ).

As I have worked through the functions, the input dimensions have increased and I have had to reduce the grid size to reduce computation requirements.

Week 2:

In Week 1, I started getting errors when executing the code for determining the best query. I had to reduce the size of the grid to get the code to run.

This week I have kept the grid size constant (i.e. 100) but changed my code to reduce the number of points of computation by down sampling the total points and by a picking points

that are apart by a factor of the square of the dimension of the input. This has reduced the computational/memory requirements and still allows me explore a large grid size.

I considered logistic regression but it is most useful when the output a clear classification, ideally between two possibilities (Yes/No, On/Off etc...)

Week 3:

I have stayed with Exploration this week because we still have a few more submission to go. As I get closer to the end, I will start exploiting. I have tried various values for ξ (PI and EI) and κ (UCB) and then selected the query with the best score.

I have increased the grid size to explore a larger area this has increased the number of computation but I have tweaked the down sampling to keep the computation requirements within reasonable limits.

In certain functions I have tried to use feature importance measure and RBF kernel to convert importance to length scales in the kernel which I passed to the Gaussian Process Regressor.

I considered SVM but that can be used only where there is a clear localisation of the optimum/maximised values. For example, **function-1** where the radiation measurement is non-zero only in the region close to the contamination source. Similarly, **function-5** where which is unimodal with a single peak where yield is maximised. SVM may not be useful in for functions like **function-3** (drug discovery) where the side-effects would change gradually or **function 4** and **function 6** where output will be improved gradually my fine tuning the hyper parameters. In essence the margins between the optimal value and others are very small.

Week 4:

I have established a flow for evaluating the grid and identifying the candidate points/queries for submission.

This week I have focused on reducing computational requirement by filtering the grid using SVM. Based on the requirement and behaviour of functions I have used, Gaussian Process, Median or K nearest neighbour techniques. I have retained my explorative stance. Filtering has allowed me to keep the exploration size large and yet reduce the number of candidate points to evaluate. This is in addition to the down sampling that I introduced in Week 2.

Week 5:

I have developed two neural networks. One that is Fixed and the other that uses gradient descent and backpropagation. Compared the results of the two. Then, to reduce the number of epochs I have introduced early stopping, the iterations stop once the loss falls below a certain threshold. I have used another value called patience that allows the iterations to continue for a fixed number of times even after the loss threshold is reached.

The steps I have introduced are

- 1) Read input data
- 2) Add the results of the previously submitted queries.
- 3) Train the surrogate on the input
- 4) Create/Prepare the grid, down sample, filter using SVM (gp, median, knn).
- 5) Predict the output for the filtered grid using the trained surrogate.
- 6) Apply Bayesian optimisation/compute acquisition for various values of ξ , κ , ρ .
- 7) Find best candidate.

Week 6:

I have made several changes to my code base. I have moved towards using a configuration file. The config file has my hyperparameters for model training like learning rate, epochs, patience etc... and hyperparameters for optimisation like ξ , κ , grid size, filter strategy etc... Then there are two config variables *optimization_direction* and *objective_mode*.

Optimisation direction has the following values

- For "*min*", switched to minimization PI/EI formulas (using y_{min} instead of y_{max}).
- For "*min*", UCB becomes a Lower Confidence Bound analogue ($-\text{mean} + \kappa * \text{std}$).

And *objective_mode* has the following values.

- Raw: Train on the actual outputs, optimise them directly.
- Zero_target: Train on squared outputs, optimise distance to zero.
- Negated: Flip the sign so that maximisation logic corresponds to minimisation.

Another general change that I have made is to the *create_bound* function such that every grid or candidate generated from these bounds will respect the constraint that individual inputs cannot be negative.

To keep record of the various runs of the model I am now recording the output candidate points and plots in the results folder with the timestamp of the run in the folder name.

Week 7:

For higher dimension functions the grid size becomes too large so I have tried using Latin Hypercube Sampling and Sobol in addition to Cartesian (the simple grid size) this week to find candidate points/queries.

I have done this introducing a config parameter called '*sample_strategy*'

sample_strategy: "*lhs*" # cartesian / latin hypercube sampling (*lhs*) / sobol

Week 8:

I have used the emergence features of the OpenAI GPT-4.1 model to get candidate points/queries this week.

I have added configuration that allows me to switch between the LLM and other models I have developed so far.

Week 9:

This week I have gone a step further in the details I have provided in the prompts. I have included specific characteristics of the function for which I am asking the candidate point. For example, for function 1, I have added the following text to the prompt

"Note that only proximity yields a non-zero reading in a contamination radiation field"

Performance:

The table below gives a summary of the results per function over the past eight weeks.

Week	Function 1	Function 2	Function 3	Function 4	Function 5	Function 6	Function 7	Function 8
1	- 1.248782933 8434956e-49	0.051646432 712366415	- 0.111185714 32117087	- 4.62023232 0523549	872.307487 3002278	- 0.53779426 02933444	1.181422704 250112	9.5658
2	- 9.343447791 332904e-112	0.180119248 02681153	- 0.030680318 237259866	- 4.93996005 1262183	743.118248 1623213	- 0.71024151 73770004	0.976917126 1966855	9.609759 2309634
3	4.507438013 8539914e-133	0.533826984 0060812	- 0.048179209 676502605	- 5.31337444 7160626	1118.87254 69008125	- 0.68332539 74004205	1.415182353 706698	9.587655 3462161
4	- 1.519971968 4953676e-55	0.596081363 5984847	- 0.020404412 90319508	- 12.8677490 36987885	386.681873 9904094	- 1.49912747 26694743	0.479035956 1505474	9.114077 0207914
5	- 6.688485279 296031e-142	0.016861726 30224915	- 0.117538864 3248932	- 7.22157012 8386034	4564.32885 7219503	- 0.88661840 75971868	0.028769356 636723045	9.292117 9841235
6	- 1.802278896 6239472e-144	0.021320835 230587712	- 0.148291020 5289665	- 7.60057727 87938355	6619.92379 5891087	- 2.03901337 91009304	0.335537295 63585927	9.567349 0736736
7	- 1.315157466 9815363e-142	0.041707825 61126679	- 0.126356164 38454833	- 4.55036289 7100278	6717.03999 0516725	- 2.30636469 75145775	0.916195395 3665816	9.514437 4297026
8	- 0.004236010 437020722	0.597629397 1773099	- 0.037692123 265023345	- 7.69091559 28365855	3942.20744 7141211	- 0.63492741 95629477	1.124185998 9248546	9.588003 1008211

Performance: summarise your results across the eight functions. What metrics did you use?

Assumptions and limitations:

One major assumption is that all the functions are continuous. In some functions that assumption is that the maxima is achieved gradually, this is not the case for each function. For example, for function1 the *"Note that only proximity yields a non-zero reading in a contamination radiation field"* suggests that there is a sharp inflection in the function in certain regions.

As the dimensions increase with each function, the exploration is limited to finding local optima due to computational limitations.

Ethical considerations:

I have documented the approach I have taken each week. Others can follow the steps outlined and use the code to duplicate my results.